

K.SUJITH 2303A52346

Batch 45

Ass 8.3

Task 1: Email Validation using TDD

Scenario

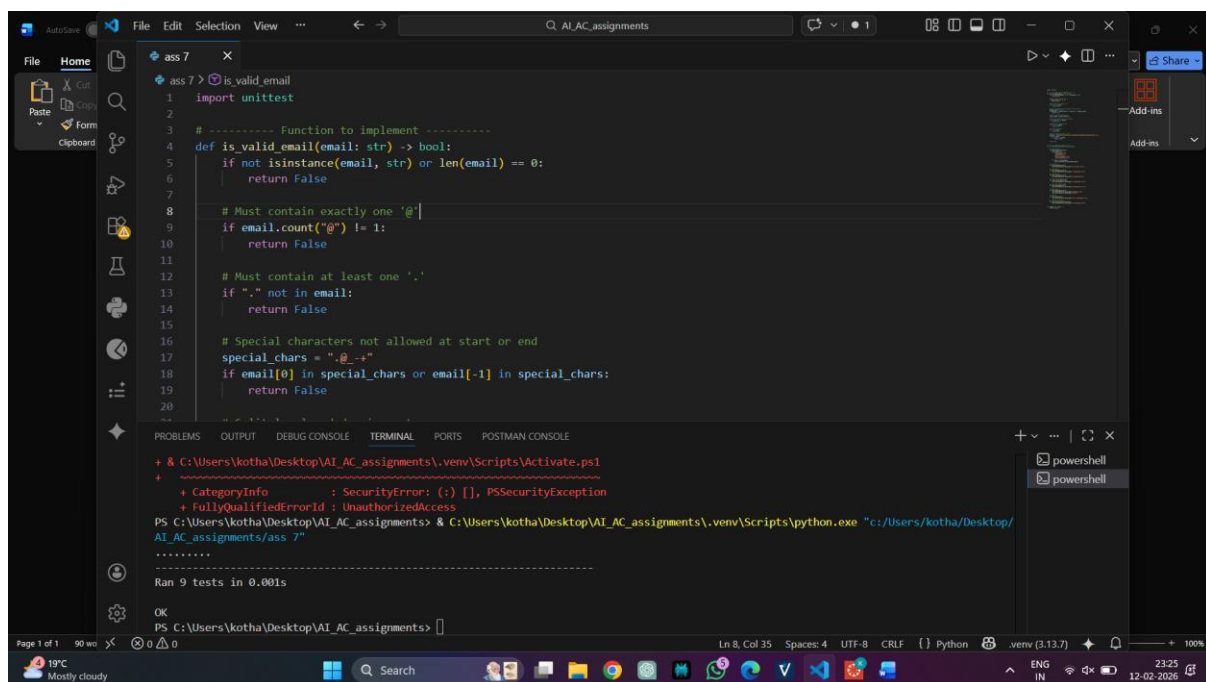
You are developing a user registration system that requires reliable email input validation.

Requirements

- Must contain @ and . characters
- Must not start or end with special characters
- Should not allow multiple @ symbols
- AI should generate test cases covering valid and invalid email formats
- Implement `is_valid_email(email)` to pass all AI-generated test cases

Expected Output

- Python function for email validation
- All AI-generated test cases pass successfully
- Invalid email formats are correctly rejected
- Valid email formats return True



The screenshot shows a VS Code editor with a Python file named `ass 7`. The code defines a function `is_valid_email(email: str) -> bool` with the following logic:

```
1 import unittest
2
3 # ----- Function to implement -----
4 def is_valid_email(email: str) -> bool:
5     if not isinstance(email, str) or len(email) == 0:
6         return False
7
8     # Must contain exactly one '@'
9     if email.count("@") != 1:
10        return False
11
12    # Must contain at least one '.'
13    if "." not in email:
14        return False
15
16    # Special characters not allowed at start or end
17    special_chars = ".@-+"
18    if email[0] in special_chars or email[-1] in special_chars:
19        return False
20
```

The terminal at the bottom shows the execution of the script using `python.exe`. It displays a `SecurityError` and `UnauthorizedAccess` message, followed by the output: `Ran 9 tests in 0.001s`. The status bar at the bottom indicates the file is in Python mode, using UTF-8 encoding, and the terminal is running in a PowerShell environment.

Task 2: Grade Assignment using Loops

Scenario

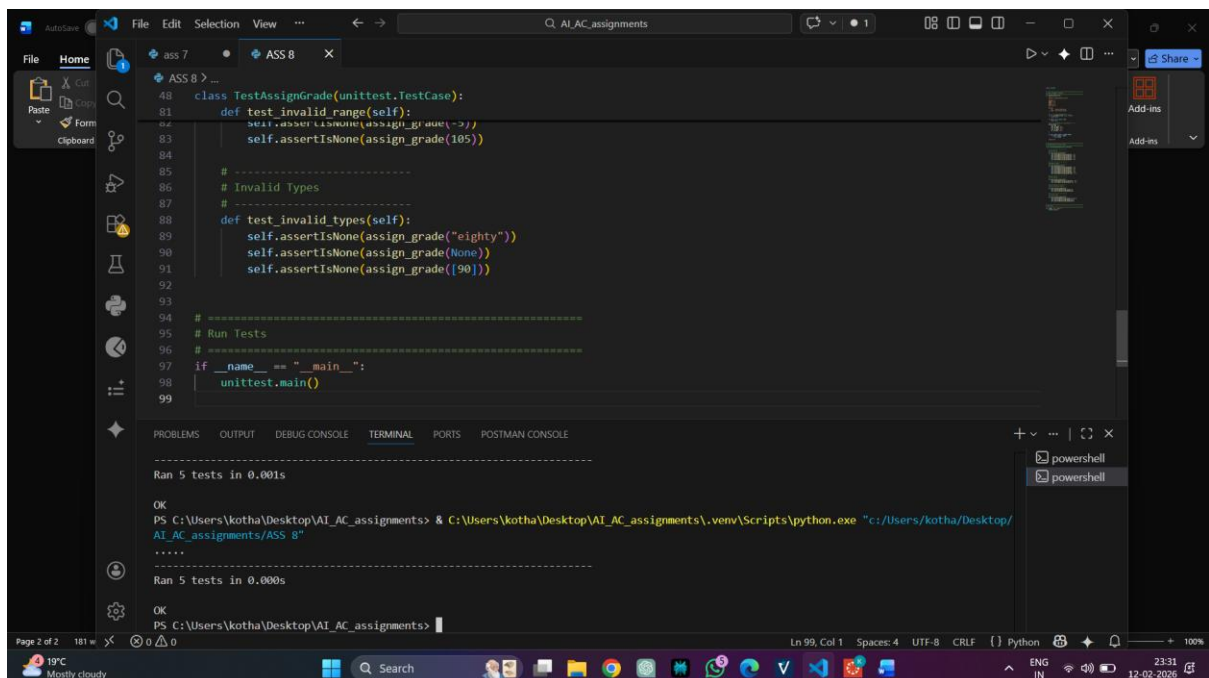
You are building an automated grading system for an online examination platform.

Requirements

- AI should generate test cases for `assign_grade(score)` where:
 - 90–100 → A
 - 80–89 → B
 - 70–79 → C
 - 60–69 → D
 - Below 60 → F
- Include boundary values (60, 70, 80, 90)
- Include invalid inputs such as -5, 105, "eighty"
- Implement the function using a test-driven approach

Expected Output

- Grade assignment function implemented in Python
- Boundary values handled correctly
- Invalid inputs handled gracefully
- All AI-generated test cases pass



```
48 class TestAssignGrade(unittest.TestCase):
49     def test_invalid_range(self):
50         self.assertRaises(ValueError, assign_grade, 105)
51
52     # Invalid Types
53     def test_invalid_types(self):
54         self.assertRaises(ValueError, assign_grade, "eighty")
55         self.assertRaises(ValueError, assign_grade, None)
56         self.assertRaises(ValueError, assign_grade, [90])
57
58 # Run Tests
59 if __name__ == "__main__":
60     unittest.main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
Ran 5 tests in 0.001s
OK
PS C:\Users\kotha\Desktop\AI_AC_assignments> & C:\Users\kotha\Desktop\AI_AC_assignments\.venv\Scripts\python.exe "c:/Users/kotha/Desktop/AI_AC_assignments/ASS_8"
.....
Ran 5 tests in 0.000s
OK
PS C:\Users\kotha\Desktop\AI_AC_assignments>
```

Task 3: Sentence Palindrome Checker

Scenario

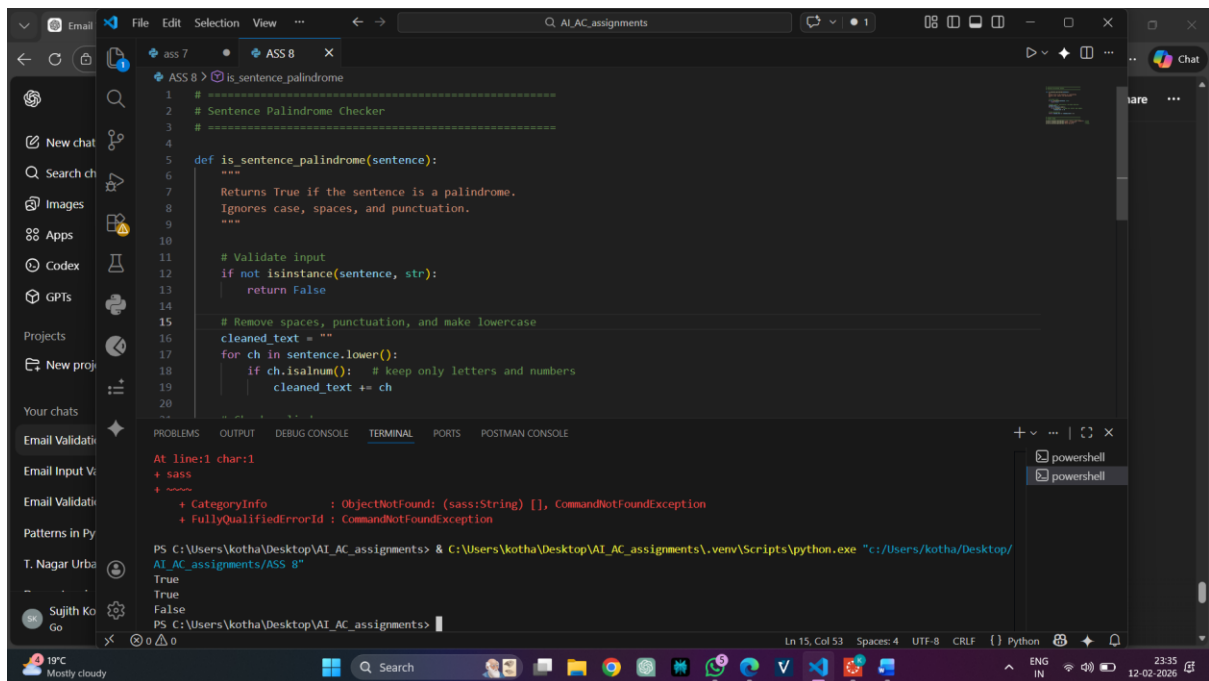
You are developing a text-processing utility to analyze sentences.

Requirements

- AI should generate test cases for `is_sentence_palindrome(sentence)`
- Ignore case, spaces, and punctuation
- Test both palindromic and non-palindromic sentences
- Example:
 - "A man a plan a canal Panama" → True

Expected Output

- Function correctly identifies sentence palindromes
- Case and punctuation are ignored
- Returns True or False accurately
- All AI-generated test cases pass



The screenshot shows a Visual Studio Code editor with a file named `ASS 8 > is_sentence_palindrome`. The code defines a function `is_sentence_palindrome(sentence)` that checks if a sentence is a palindrome, ignoring case, spaces, and punctuation. The function uses `isinstance` to validate input, strips non-alphanumeric characters, and compares the cleaned string with its reverse. The terminal shows the function being called with the example sentence, returning `True`, and then with a non-palindrome, returning `False`.

```
1 # =====  
2 # Sentence Palindrome Checker  
3 # =====  
4  
5 def is_sentence_palindrome(sentence):  
6     """  
7     Returns True if the sentence is a palindrome.  
8     Ignores case, spaces, and punctuation.  
9     """  
10  
11     # Validate input  
12     if not isinstance(sentence, str):  
13         return False  
14  
15     # Remove spaces, punctuation, and make lowercase  
16     cleaned_text = ""  
17     for ch in sentence.lower():  
18         if ch.isalnum(): # keep only letters and numbers  
19             cleaned_text += ch  
20  
21     return cleaned_text == cleaned_text[::-1]
```

Terminal output:

```
PS C:\Users\kotha\Desktop\AI_AC_assignments> & C:\Users\kotha\Desktop\AI_AC_assignments\.venv\Scripts\python.exe "c:/Users/kotha/Desktop/AI_AC_assignments/ASS 8"  
True  
True  
False  
PS C:\Users\kotha\Desktop\AI_AC_assignments>
```

Task 4: ShoppingCart Class

Scenario

You are designing a basic shopping cart module for an e-commerce application.

Requirements

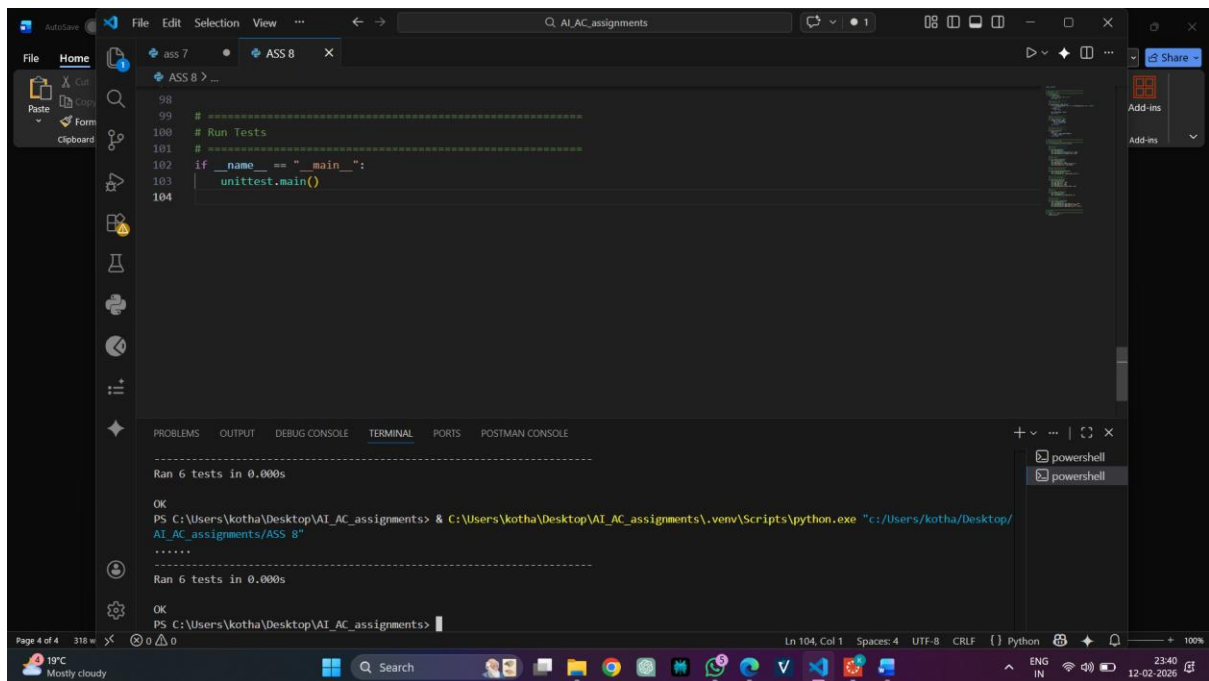
- AI should generate test cases for the `ShoppingCart` class
- Class must include the following methods:
 - `add_item(name, price)`
 - `remove_item(name)`

– total_cost()

- Validate correct addition, removal, and cost calculation
- Handle empty cart scenarios

Expected Output

- Fully implemented ShoppingCart class
- All methods pass AI-generated test cases
- Total cost is calculated accurately
- Items are added and removed correctly



```
98
99 # Run Tests
100 # Run Tests
101 # Run Tests
102 if __name__ == "__main__":
103     unittest.main()
104
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

Ran 6 tests in 0.000s

OK

PS C:\Users\kotha\Desktop\AI_AC_assignments> & C:\Users\kotha\Desktop\AI_AC_assignments\.venv\Scripts\python.exe "c:/Users/kotha/Desktop/AI_AC_assignments/ASS_8"

.....

Ran 6 tests in 0.000s

OK

PS C:\Users\kotha\Desktop\AI_AC_assignments>

Task 5: Date Format Conversion

Scenario

You are creating a utility function to convert date formats for reports.

Requirements

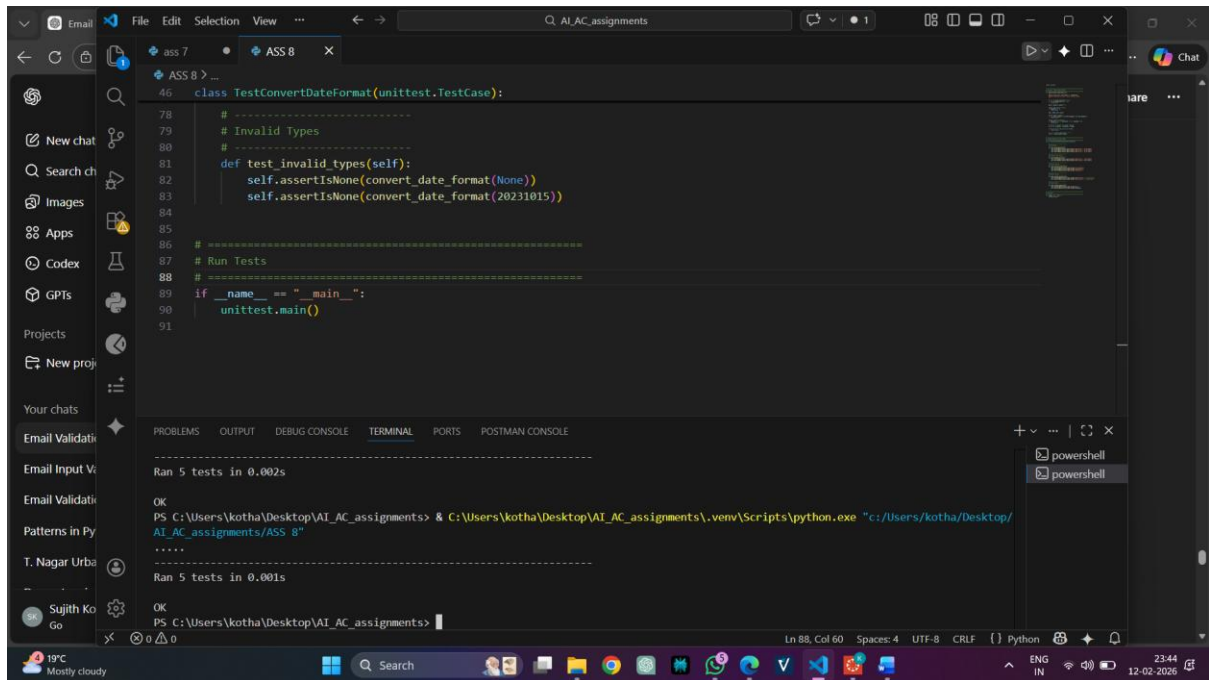
- AI should generate test cases for convert_date_format(date_str)
- Input format must be "YYYY-MM-DD"
- Output format must be "DD-MM-YYYY"
- Example:

– "2023-10-15" → "15-10-2023"

Expected Output

- Date conversion function implemented in Python

- Correct format conversion for all valid inputs
- All AI-generated test cases pass successfully



The screenshot displays the Visual Studio Code (VS Code) interface. The main editor window shows a Python file named `ASS 8` with the following code:

```
46 class TestConvertDateFormat(unittest.TestCase):
78     # -----
79     # Invalid Types
80     # -----
81     def test_invalid_types(self):
82         self.assertIsNone(convert_date_format(None))
83         self.assertIsNone(convert_date_format(20231015))
84
85     # -----
86     # Run Tests
87     # -----
88     # -----
89     if __name__ == "__main__":
90         unittest.main()
91
```

The bottom panel of VS Code shows the `TERMINAL` tab with the following output:

```
-----
Ran 5 tests in 0.002s

OK
PS C:\Users\kotha\Desktop\AI_AC_assignments> & C:\Users\kotha\Desktop\AI_AC_assignments\.venv\Scripts\python.exe "c:/Users/kotha/Desktop/AI_AC_assignments/ASS 8"
-----
Ran 5 tests in 0.001s

OK
PS C:\Users\kotha\Desktop\AI_AC_assignments>
```

The terminal output confirms that all 5 tests passed successfully, resulting in an `OK` status.